

Key considerations about **SQL Server AUTOGROW and AUTOSHRINK settings** from the Microsoft documentation for quick reference:

Source:

<https://learn.microsoft.com/en-us/troubleshoot/sql/database-engine/database-file-operations/considerations-autogrow-autoshrink>

Considerations: AUTOGROW and AUTOSHRINK (SQL Server)

General Configuration

- Autogrow and autoshrink settings can be configured via SSMS or ALTER DATABASE.
- These settings apply **per file** (data and log); you must set them on **each file individually**.

AUTO_SHRINK Considerations

- **AUTO_SHRINK** enables a background task to shrink data/log files automatically.
- Frequent shrink and grow cycles lead to **file system fragmentation**, negatively impacting performance.
- Shrinking a file may cause later operations to be slower if space must be regrown.
- The shrink background task consumes resources and may interfere with normal workload.
- It acquires locks and synchronization that can conflict with active processes.
- Best practice: avoid autoshrink; provision database files to required size instead.

AUTOGROW Considerations

- Autogrow is useful as a **contingency** but not ideal for day-to-day growth management.
- If a transaction requires more log space, it must wait for the file to grow, which **adds to transaction time**.
- Large or frequent autogrow events can introduce **performance penalties** and potential timeouts.
- Repeated growth of log files can create many **Virtual Log Files (VLFs)**, slowing operations like startup, replication, mirroring, CDC, and log backups.
- Combining autogrow with autoshrink can cause excessive overhead, with files growing then shrinking repeatedly.
- Small growth increments increase fragmentation and reduce performance; larger, planned increments are recommended.

Best Practices

- Treat autogrow as a **safety net**, not a primary method for space management.
- Proactively monitor file sizes and grow them manually as part of maintenance.
- Use alerts/monitoring tools to anticipate file growth needs.
- Carefully choose growth increments (fixed MB rather than percentages where appropriate) to reduce performance costs.
- Set a **MAXSIZE** to avoid files consuming all available disk space unexpectedly.
- Minimize transaction sizes where possible to reduce unplanned growth.

Recommended configuration and best practice guide for AUTOGROW settings in SQL Server, especially for OLTP or large-scale environments:

Recommended AUTOGROW Settings

1. Data Files (MDF/NDF)

- **Growth Type:** Prefer **fixed size (MB)** rather than percentage.
- **Size Recommendations:**
 - Small databases: 64–256 MB per growth increment.
 - Medium/large databases: 512 MB–1 GB per growth increment.
- **Reason:** Fixed size avoids repeated small autogrows, reduces fragmentation, and improves performance.
- **MAXSIZE:** Set a sensible maximum to avoid consuming all disk space unexpectedly.

2. Log Files (LDF)

- **Growth Type:** Always **fixed size (MB)**.
- **Size Recommendations:**
 - Small logs: 64–128 MB increments.
 - Large logs: 256–512 MB increments.
- **Reason:** Percentage growth creates too many VLFs in large log files, hurting performance.
- **MAXSIZE:** Limit to available disk space and anticipated growth.

3. Avoid AUTOSHRINK

- Do **not enable autoshrink**, as it causes repeated shrink/grow cycles, fragmentation, and performance overhead.
- Shrinking files should only be done **manually** during maintenance windows if needed.

4. Monitoring & Maintenance

- Regularly monitor **file sizes, disk usage, and VLF count**.
- Plan manual growth ahead of peak workloads instead of relying on autogrow.
- Use alerts to catch near-full disk conditions or rapid growth patterns.

5. Example: Recommended SQL Commands

-- Set fixed autogrow for data file

```
ALTER DATABASE MyDB
MODIFY FILE
```

```
(
    NAME = MyDB_Data,
    FILEGROWTH = 256MB,
    MAXSIZE = 102400MB
```

```
);
```

-- Set fixed autogrow for log file

```
ALTER DATABASE MyDB
MODIFY FILE
```

```
(
    NAME = MyDB_Log,
    FILEGROWTH = 128MB,
    MAXSIZE = 51200MB
```

```
);
```

6. Key Takeaways

- AUTOGROW = **safety net**, not primary growth strategy.
- Fixed-size growth reduces fragmentation and VLF count.
- Avoid AUTOSHRINK entirely.
- Monitor disk space proactively and grow files manually as needed.

Here's a **quick visual table** summarizing **Data vs Log files autogrow settings and best practices** for SQL Server:

File Type	Growth Type	Recommended Increment	MAXSIZE	Notes / Best Practices
Data (MDF/NDF)	Fixed size (MB)	Small DB: 64–256 MB Medium/Large DB: 512 MB–1 GB	Set according to disk space	Avoid percentage growth; reduces fragmentation and frequent autogrow events
Log (LDF)	Fixed size (MB)	Small log: 64–128 MB Large log: 256–512 MB	Limit to available disk	Prevents too many VLFs; avoids transaction slowdowns
Both	Autoshrink	Disable	N/A	Causes repeated shrink/grow cycles, fragmentation, and performance overhead
Monitoring	Manual / Alerts	N/A	N/A	Regularly monitor file size, disk space, and growth trends; plan manual growth during maintenance windows
Key Tip	AUTOGROW	Safety net only	N/A	Do not rely on autogrow for daily operations; plan file sizes proactively

This table can be included in **internal documentation, team presentations, or DBA SOPs** to quickly communicate the recommended strategy for SQL Server file growth.

SQL Server

Autogrow & Autoshrink Best Practices



DATA FILES
(MDF / NDF)



LOG FILES
(LDF)

Recommended Settings

DATA FILES (MDF / NDF)

- **Growth Type:** Fixed Size (MB)
- **Increment:** 256 MB – 1 GB

LOG FILES (LDF)

- **Growth Type:** Fixed Size (MB)
- **Increment:** 128 MB – 512 MB

Avoid Autoshrink!



- Disable Autoshrink
- Causes Fragmentation
- Hurts Performance

Best Practices



Monitor & Alert
on Disk Space



Manually Grow Files
During Maintenance



Set MAXSIZE to
Prevent Overgrowth

Autogrow = Safety Net, Not a Strategy

hamps.com/